

C.P.R. Liceo “La Paz”

Proyecto Fin de Ciclo

Desarrollo de Aplicaciones Multiplataforma

Autor: Julio Valcárcel Vigier Tutor: Jesús Ángel Pérez-Roca Fernández

FitProgress

• Resumen

FitProgress es una aplicación móvil desarrollada con React Native y Expo en el frontend, y Node.js con Express en el backend, dirigida a personas que entrenan en el gimnasio de forma autónoma y necesitan una herramienta para organizar sus rutinas y registrar su evolución física.

La aplicación permite al usuario crear rutinas de entrenamiento personalizadas organizadas por días de la semana, donde puede definir los ejercicios, el número de series, las repeticiones objetivo y el tiempo de descanso entre series. Durante el entrenamiento, la app ofrece una pantalla en vivo donde se registra cada serie con el peso utilizado y las repeticiones realizadas, incluyendo un temporizador de descanso configurable que alerta mediante vibración cuando finaliza.

El catálogo incluye 47 ejercicios predefinidos con vídeos demostrativos de YouTube, organizados por grupo muscular (pecho, espalda, hombros, brazos, piernas, core y cardio), y el usuario puede crear sus propios ejercicios personalizados y marcar cualquiera como favorito para un acceso más rápido.

La app cuenta con un sistema de seguimiento corporal que permite registrar medidas como peso, porcentaje de grasa, y circunferencias de pecho, cintura, brazo y pierna, mostrando la evolución mediante gráficos de barras. También ofrece un historial completo de todas las sesiones realizadas con el detalle de cada ejercicio, serie y volumen total.

El proyecto utiliza PostgreSQL como sistema de gestión de base de datos, con 8 tablas que cubren usuarios, ejercicios, rutinas, sesiones, series, medidas corporales y favoritos. La autenticación se gestiona mediante tokens JWT y el cifrado de contraseñas con bcrypt. La comunicación entre el frontend y el backend se realiza a través de una API REST con 6 grupos de endpoints.

La aplicación funciona en dispositivos Android e iOS a través de Expo Go, y también es compatible con navegadores web.

• Abstract

FitProgress is a cross-platform mobile application built with React Native and Expo on the frontend, and Node.js with Express on the backend, designed for people who train independently at the gym and need a practical tool to manage their workout routines and track their physical progress over time.

The application allows users to create fully customisable training routines organised by day of the week, specifying exercises, target sets, repetitions and rest time between sets. During a workout session, the app provides a live training screen where each set is logged with the weight lifted and repetitions performed, alongside a configurable rest timer that notifies the user through vibration when the break is over.

The exercise catalogue includes 47 predefined exercises with YouTube demonstration videos, sorted by muscle group (chest, back, shoulders, arms, legs, core and cardio). Users can also create their own custom exercises and mark any exercise as a favourite for quicker access.

A body tracking system is integrated into the app, enabling users to log measurements such as weight, body fat percentage, and chest, waist, arm and leg circumferences. The recorded data is displayed through bar charts that show the user's progress over time. Additionally, a full history of completed training sessions is available, including detailed breakdowns of every exercise, set and total volume.

The project relies on PostgreSQL as the database management system, with 8 tables covering users, exercises, routines, sessions, sets, body measurements and favourites. User authentication is handled through JWT tokens and password encryption with bcrypt. Communication between the frontend and the backend is carried out via a REST API with 6 endpoint groups.

The application runs on both Android and iOS devices through Expo Go, and is also compatible with web browsers.

• Palabras Clave

• **React Native:** *Framework* de desarrollo de código abierto empleado para construir la interfaz de usuario del cliente móvil, permitiendo generar aplicaciones multiplataforma (iOS y Android) de forma nativa a partir de una única base de código.

• **Expo:** Conjunto de herramientas y servicios construidos sobre React Native que simplifican el desarrollo, la compilación, la prueba en dispositivos físicos y el despliegue de aplicaciones móviles nativas.

• **Node.js:** Entorno de ejecución asíncrono y orientado a eventos, utilizado para implementar la lógica de negocio del servidor (*backend*), capaz de gestionar múltiples conexiones concurrentes de forma eficiente.

• **Express.js:** Infraestructura web minimalista y flexible para Node.js, utilizada para la creación del servidor, la gestión de *middlewares* y el enrutamiento eficiente de las peticiones HTTP.

• **API RESTful:** Estilo de arquitectura de *software* basado en el protocolo HTTP que define un conjunto de reglas y convenciones para la comunicación estandarizada, escalable y desacoplada entre la aplicación móvil y el servidor.

• **PostgreSQL:** Sistema de gestión de bases de datos relacional orientado a objetos (ORDBMS), de código abierto y alta robustez, empleado para el almacenamiento persistente e íntegro de la información de la aplicación.

• **Prisma ORM:** Herramienta avanzada de mapeo objeto-relacional (*Object-Relational Mapping*) empleada para interactuar con la base de datos PostgreSQL, facilitando el modelado de esquemas, las migraciones y garantizando el acceso a datos con seguridad de tipos (*type-safety*).

• **TypeScript:** Superconjunto sintáctico del lenguaje JavaScript utilizado en todo el ecosistema del proyecto (cliente y servidor) que añade tipado estático, reduciendo la incidencia de errores en tiempo de ejecución y mejorando la mantenibilidad del código.

• **Autenticación *Stateless* (JWT):** Mecanismo de seguridad "sin estado" implementado mediante *JSON Web Tokens*, donde la validación de la sesión del usuario se realiza criptográficamente a través del *token*, eliminando la necesidad de almacenar el estado de la sesión en la memoria del servidor.

• **Cifrado Hash (Bcrypt):** Algoritmo criptográfico unidireccional utilizado para ofuscar las contraseñas de los usuarios antes de su almacenamiento en la base de datos, mitigando riesgos de brechas de seguridad y garantizando altos estándares de ciberseguridad.

• **Arquitectura Cliente-Servidor:** Patrón de diseño estructural que divide la aplicación en dos componentes independientes que se comunican a través de una

red: el cliente (encargado de la presentación e interacción) y el servidor (encargado del procesamiento y almacenamiento centralizado de datos).

- **Patrón MVC (Modelo-Vista-Controlador):** Patrón de arquitectura de *software* aplicado en el diseño del sistema que separa de forma lógica la representación de la información (Vista), la gestión de los datos y reglas de negocio (Modelo) y el enrutamiento de las peticiones (Controlador).

- **Middleware:** Componente de *software* intermedio que se ejecuta durante el ciclo de vida de una petición HTTP en el servidor, utilizado en este proyecto para tareas transversales como la validación estricta de datos entrantes y la verificación de credenciales de seguridad.

- **JSON (JavaScript Object Notation):** Formato de texto ligero y estructurado para el intercambio de datos, empleado como estándar de comunicación bidireccional entre el cliente móvil (la vista) y la API del servidor.

- **React Hooks:** Funciones integradas en la biblioteca React (como `useState` o `useEffect`) que permiten a los componentes funcionales interactuar con el estado interno y los ciclos de vida de la aplicación sin necesidad de recurrir a la programación basada en clases.

- **Gestión de Estado Global:** Técnica de ingeniería de *software* utilizada en el *frontend* (mediante *React Context API*) para compartir y administrar información persistente accesible desde múltiples vistas de la aplicación, como la sesión activa del usuario o la configuración del tema visual (claro/oscuro).

FitProgress

Dedicatoria o agradecimientos.

A mis padres, por su apoyo incondicional, su paciencia y su sacrificio diario, pilares fundamentales que han guiado cada uno de mis pasos no solo a lo largo de este ciclo formativo, sino en cada etapa de mi vida.

Al cuerpo docente del C.P.R. Liceo La Paz, por su dedicación, su rigor académico y por transmitirnos los conocimientos y valores necesarios para afrontar el mundo laboral con solvencia.

A mis compañeros de aula, con quienes he compartido extensas jornadas de programación, debates arquitectónicos y una colaboración mutua que ha hecho de este proceso de aprendizaje una experiencia sumamente enriquecedora.

Y, por último, a todas aquellas personas que, de forma directa o indirecta, depositaron su confianza en mí y contribuyeron con su aliento a la culminación con éxito de este proyecto.

FitProgress

Sumario

Resumen 3
Abstract 4
Palabras Clave 5
Introducción/motivación 11
Objetivos 12
Estado del arte 13
Caso de estudio 14
Diagramas 15
Desarrollo del proyecto 16
Manual del Administrador 17
Manual del Usuario 18
Forma jurídica y trámites de constitución 19
Viabilidad tecno-económica 20
Trabajo futuro 21
Conclusiones 22
Biblioteca de recursos web y referencias 23
Anexos 24

• Introducción/motivación

En los últimos años, la concienciación social en torno a la salud física, el bienestar y la práctica estructurada del entrenamiento de fuerza ha experimentado un crecimiento sin precedentes. La optimización del rendimiento en el gimnasio y la consecución de objetivos como la hipertrofia muscular o la ganancia de fuerza no dependen únicamente del esfuerzo físico aislado, sino de la aplicación estricta del principio de sobrecarga progresiva. Para cumplir este principio, es biológicamente indispensable registrar con precisión matemática variables como las cargas levantadas, las repeticiones efectuadas, el volumen total de trabajo y los tiempos de recuperación interserie.

La elección de desarrollar **FitProgress** como Proyecto Fin de Ciclo, en detrimento de otras alternativas tradicionales como plataformas de comercio electrónico o videojuegos convencionales, se fundamenta en un doble orden de motivaciones:

Por un lado, existe una motivación de índole funcional y de mercado. Tras analizar el estado del arte actual, se constató que la gran mayoría de las herramientas existentes sufren de una monetización sumamente restrictiva o de interfaces de usuario excesivamente complejas que entorpecen la fluidez del entrenamiento en el propio gimnasio. Muchas de estas aplicaciones bloquean el acceso al historial detallado o al cálculo automatizado del volumen de entrenamiento tras muros de pago recurrentes. Como entusiasta del ámbito del acondicionamiento físico, detecté una oportunidad clara para diseñar y desarrollar una solución de software que fuera completamente gratuita, que estuviera libre de publicidad intrusiva y que proporcionara métricas avanzadas de análisis de fuerza de forma abierta y directa.

Por otro lado, se presenta una profunda motivación técnica ligada a las competencias del Ciclo Formativo de Grado Superior en Desarrollo de Aplicaciones Multiplataforma. El desarrollo de una aplicación de gestión deportiva con procesamiento analítico de datos en tiempo real constituía un escenario técnico ideal para abordar un proyecto *Full-Stack* de alta complejidad. Este desarrollo exigía integrar de forma nativa la persistencia de datos relacionales interconectados, la seguridad en las comunicaciones a través de *tokens* criptográficos y la creación de una experiencia móvil multiplataforma fluida, reactiva y atractiva (incorporando la gestión de temas dinámicos oscuro/claro mediante el estado global). La resolución de sincronizaciones asíncronas entre la interfaz y el servidor durante el registro de datos supuso un reto arquitectónico sumamente enriquecedor para mi perfil profesional.

A través de la implementación de este proyecto, se persigue alcanzar un conjunto de metas tanto funcionales como tecnológicas que resuelvan las necesidades del usuario final y demuestren la solidez del software desarrollado:

- **Proporcionar una experiencia de usuario fluida y optimizada:** Diseñar una interfaz interactiva móvil que permita registrar series de ejercicios con un mínimo de pulsaciones en pantalla, reduciendo las distracciones durante la sesión deportiva.

- **Automatizar el análisis analítico del rendimiento:** Delegar en el servidor el cálculo algorítmico automatizado del volumen total de entrenamiento por sesión (peso × repeticiones) y de las cargas máximas alcanzadas por ejercicio, transformando los datos brutos en gráficos evolutivos comprensibles.
- **Garantizar la persistencia y la integridad de la información:** Consolidar una base de datos relacional robusta en PostgreSQL que preserve sin fisuras el historial de entrenamientos, las rutinas planificadas y la evolución biométrica (peso y perímetros corporales) del usuario.
- **Implementar un sistema seguro y escalable:** Proteger el acceso a la plataforma mediante protocolos de autenticación estandarizados en la industria (JWT) y aplicar técnicas avanzadas de validación y control de errores en el servidor para asegurar la resiliencia del ecosistema informático.

• Objetivos

La concepción y el desarrollo de *FitProgress* se vertebran en torno a una meta principal que define el propósito del proyecto, la cual se desglosa en una serie de metas técnicas y funcionales orientadas a cubrir todas las fases del ciclo de vida del *software*.

Objetivo General Diseñar, desarrollar y desplegar un sistema informático integral, compuesto por una aplicación móvil multiplataforma y una API RESTful, orientado a la gestión, monitorización en tiempo real y análisis estadístico del entrenamiento de fuerza y el progreso físico del usuario.

Objetivos Específicos Para alcanzar el objetivo general propuesto, se han establecido los siguientes objetivos específicos, divididos por áreas arquitectónicas:

Objetivos vinculados al Backend y Base de Datos:

- **Modelar una base de datos relacional escalable:** Diseñar un esquema en PostgreSQL, gestionado mediante Prisma ORM, capaz de interrelacionar eficientemente perfiles de usuario, catálogos de ejercicios, rutinas estructuradas, registros de series y medidas antropométricas.
- **Desarrollar una API RESTful segura y eficiente:** Construir la lógica del servidor en Node.js y Express (con TypeScript), implementando operaciones CRUD y garantizando tiempos de respuesta óptimos.
- **Implementar protocolos de ciberseguridad:** Proteger la integridad de los datos mediante un sistema de autenticación *stateless* basado en JSON Web Tokens (JWT) y el cifrado de credenciales de acceso mediante el algoritmo *bcrypt*.
- **Garantizar la integridad de los datos entrantes:** Desarrollar *middlewares* de validación estricta en los *endpoints* del servidor para prevenir la inyección de datos maliciosos o inconsistentes.

Objetivos vinculados al Frontend e Interfaz de Usuario:

- **Construir un cliente móvil multiplataforma:** Desarrollar la aplicación utilizando React Native y Expo, asegurando compatibilidad nativa y un rendimiento fluido tanto en dispositivos Android como iOS a partir de una única base de código.
- **Diseñar una experiencia de usuario (UX) inmersiva:** Crear una interfaz de usuario accesible e intuitiva que incluya navegación por pestañas y gestión de estado global, implementando un diseño oscuro (*dark mode*) nativo que reduzca la fatiga visual.
- **Optimizar el registro asíncrono de entrenamientos:** Desarrollar un módulo de sesión activa que permita al usuario registrar cargas, repeticiones e indicadores de fallo muscular en tiempo real, integrando temporizadores automáticos para los descansos interserie.

Objetivos vinculados a la Analítica y Funcionalidad:

- **Procesar y visualizar estadísticas de progreso:** Implementar algoritmos de agregación en el servidor que calculen el volumen total de entrenamiento y las cargas máximas, mostrando esta información en la aplicación móvil a través de gráficas y tarjetas de resumen.
- **Facilitar la personalización del plan de entrenamiento:** Permitir a los usuarios no solo utilizar una biblioteca predefinida, sino también crear ejercicios personalizados, duplicar rutinas y archivar planificaciones para un control total de su ciclo deportivo.

• Estado del arte

El mercado actual de aplicaciones orientadas a la salud, el *fitness* y el registro de entrenamientos es altamente competitivo, contando con soluciones maduras y ampliamente adoptadas a nivel global. Para contextualizar el desarrollo de *FitProgress*, resulta indispensable analizar tanto las alternativas de *software* existentes como las tecnologías disponibles para su implementación.

Análisis de aplicaciones similares en el mercado En la actualidad, aplicaciones como *Strong*, *Hevy* y *Jefit* dominan el sector del seguimiento de rutinas de fuerza.

- **Puntos fuertes:** Estas plataformas destacan por poseer bases de datos de ejercicios sumamente extensas (a menudo respaldadas por animaciones o vídeos) y por contar con grandes comunidades de usuarios que permiten compartir rutinas. Además, presentan interfaces gráficas muy pulidas fruto de años de iteraciones.
- **Puntos débiles:** Su principal deficiencia radica en los modelos de negocio agresivos (*freemium*). Funcionalidades esenciales para un seguimiento riguroso, como la creación de rutinas ilimitadas, la exportación de datos o el acceso a gráficos estadísticos del volumen muscular, se encuentran bloqueadas tras suscripciones de pago. Adicionalmente, el afán por incluir características sociales a menudo sobrecarga la interfaz de usuario, dificultando el registro rápido de las series mientras el deportista está entrenando.
- **Comparativa con el proyecto:** *FitProgress* se desmarca de estos competidores al ofrecer un ecosistema donde todas las herramientas analíticas (evolución de peso, volumen por sesión y cargas máximas) están centralizadas y accesibles sin restricciones. Su interfaz ha sido diseñada específicamente para minimizar la fricción durante la sesión de entrenamiento, priorizando la usabilidad mediante temporizadores integrados y flujos de registro optimizados.

Tecnologías utilizadas y posibles alternativas Para la construcción de este sistema multiplataforma, se ha seleccionado un conjunto de tecnologías (*stack*) modernas y eficientes, evaluadas frente a diversas alternativas del mercado:

1. **Frontend (Aplicación Móvil):** Se ha optado por **React Native** bajo el entorno de **Expo**.
    - *Alternativas evaluadas:* El desarrollo nativo puro (utilizando *Swift* para iOS y *Kotlin* para Android) ofrecería el máximo rendimiento posible, pero duplicaría los tiempos de desarrollo y la base de código. Otra alternativa sólida es *Flutter* (desarrollado por Google); sin embargo, la elección de React Native se justifica por su madurez, la gran disponibilidad de librerías en el ecosistema de JavaScript/TypeScript y la facilidad para gestionar estados globales mediante *Context API*.
2. **Backend (Servidor y API):** La lógica de negocio se ha implementado mediante **Node.js** y **Express.js**, empleando **TypeScript**.

- *Alternativas evaluadas:* Se contempló el uso de *Backend as a Service* (BaaS) como *Firebase* o *Supabase*, que agilizan enormemente el desarrollo inicial. No obstante, se descartaron en favor de un servidor propio en Node.js para garantizar un control absoluto sobre el enrutamiento, la seguridad (*middlewares* de validación) y la escalabilidad de la API RESTful. Otras opciones como *Django* (Python) o *Spring Boot* (Java) son igualmente válidas, pero Node.js permite unificar el lenguaje de programación (TypeScript) en ambas capas del proyecto.

3. **Persistencia de Datos:** Se ha implementado una base de datos relacional **PostgreSQL** gestionada a través de **Prisma ORM**.

- *Alternativas evaluadas:* El uso de bases de datos NoSQL como *MongoDB* ofrece gran flexibilidad para estructuras de datos dinámicas. Sin embargo, dada la naturaleza altamente estructurada de *FitProgress* donde rutinas, ejercicios, series y usuarios mantienen relaciones estrictas y dependientes, una base de datos relacional (SQL) como PostgreSQL resulta imperativa para garantizar la integridad referencial. El uso de Prisma ORM frente a alternativas como *TypeORM* o *Sequelize* aporta una seguridad de tipos (*type-safety*) superior y migraciones de base de datos más predecibles.

• Caso de estudio

Tras el análisis del estado del arte, en el que se evidenció que las soluciones actuales del mercado presentan carencias ligadas a la monetización restrictiva y a interfaces sobrecargadas, *FitProgress* se plantea como un caso de estudio orientado a democratizar y optimizar el registro del rendimiento deportivo.

El presente proyecto aborda esta problemática mediante el desarrollo de una solución integral que mejora las alternativas existentes a través de los siguientes puntos clave de implementación:

1. Democratización de los datos analíticos y estadísticos A diferencia de las aplicaciones comerciales que bloquean la visualización del progreso tras suscripciones *premium*, el desarrollo de *FitProgress* contempla un módulo estadístico abierto y procesado íntegramente en el servidor. El sistema calculará y mostrará de forma nativa métricas complejas, tales como la evolución gráfica del peso corporal, la progresión de cargas máximas por ejercicio y el volumen total de entrenamiento (calculado mediante la fórmula *peso* × *repeticiones* a lo largo de las series de una sesión).

2. Optimización del flujo de registro en tiempo real (UX/UI) Uno de los mayores obstáculos para los usuarios durante el entrenamiento es la fricción al interactuar con el dispositivo móvil. Para solucionarlo, el proyecto desarrolla una interfaz de usuario minimalista y de alto contraste (tema oscuro nativo) que ahorra batería y reduce la fatiga visual. Funcionalmente, la pantalla de entrenamiento permite registrar de manera asíncrona y con un mínimo de interacciones cada serie completada, indicando el peso, las repeticiones y la proximidad al fallo muscular. Además, incorpora un temporizador de descanso automatizado con retroalimentación háptica (vibración) que alerta al usuario cuando es momento de retomar la actividad, eliminando la necesidad de aplicaciones de terceros.

3. Flexibilidad absoluta en la planificación deportiva Las soluciones actuales suelen limitar el número de rutinas personalizadas que un usuario gratuito puede crear. Como mejora directa, *FitProgress* desarrolla un motor de planificación sin restricciones que permite:

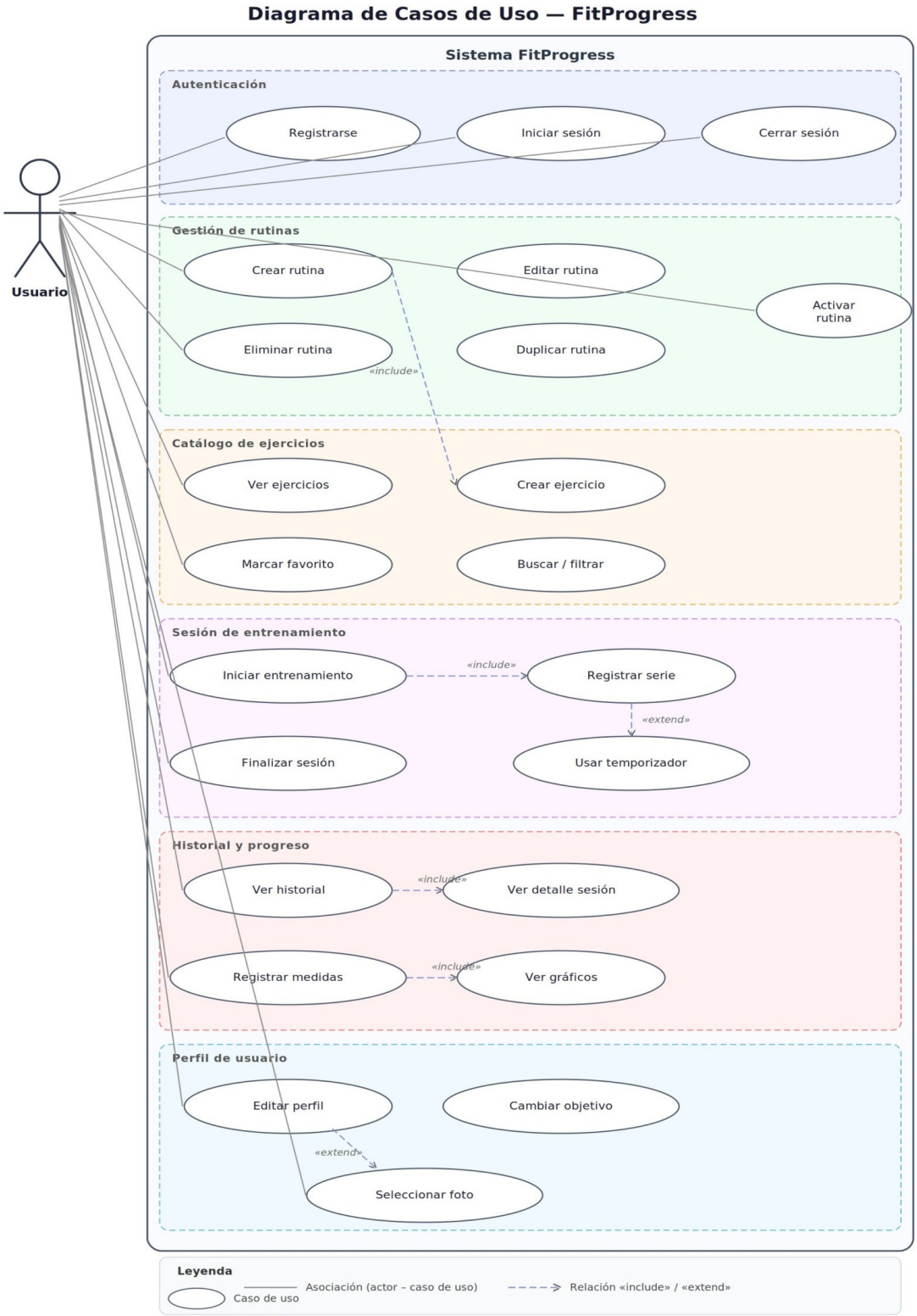
- Crear rutinas complejas asignando ejercicios a días específicos de la semana.
- Archivar rutinas antiguas para mantener el panel de control limpio sin perder el historial de datos.
- Duplicar planificaciones existentes para generar progresiones o variaciones del entrenamiento.
- Añadir ejercicios completamente personalizados al catálogo global del usuario si el movimiento deseado no se encuentra en la base de datos predefinida del sistema.

4. Arquitectura de datos relacional y escalable Para sustentar las funcionalidades

anteriores, el proyecto desarrolla una arquitectura de datos estrictamente relacional en PostgreSQL. A diferencia de soluciones basadas en almacenamiento local (*offline*), *FitProgress* asegura que la información del usuario esté sincronizada en la nube, garantizando la integridad referencial entre el historial de sesiones pasadas y los ejercicios. Esta arquitectura desacoplada Cliente-Servidor no solo resuelve el problema inmediato del registro, sino que sienta las bases tecnológicas para futuras expansiones del proyecto de forma escalable y segura.

• Diagramas

Diagrama de Casos de Uso



FitProgress

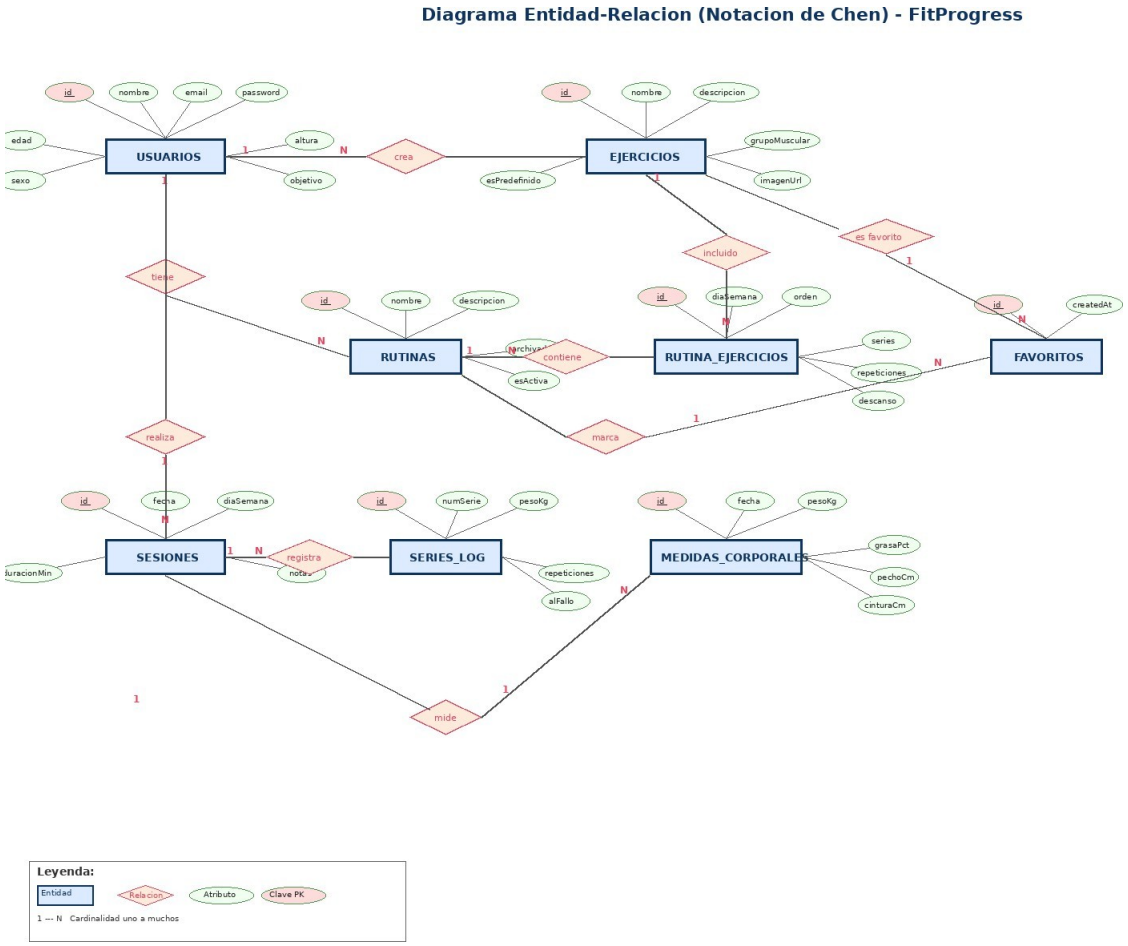
El diagrama de casos de uso define el comportamiento funcional y los límites del sistema FitProgress desde la perspectiva de las interacciones con el usuario final. Al tratarse de una aplicación de uso personal, el modelo se simplifica en torno a un único actor principal: el Usuario, quien desencadena todas las acciones del sistema.

Para garantizar una arquitectura limpia y modular, las funcionalidades se han agrupado lógicamente en seis grandes bloques operativos:

- **Autenticación:** Representa la barrera de entrada y seguridad del sistema. Engloba los procesos básicos de Registrarse, Iniciar sesión y Cerrar sesión.
- **Gestión de rutinas:** Constituye el núcleo de planificación. Permite al usuario realizar operaciones completas de control sobre sus entrenamientos (Crear, Editar, Duplicar, Eliminar y Activar rutina). Cabe destacar la dependencia **<<include>>** que parte de Crear rutina hacia Crear ejercicio, lo que indica que, dentro del flujo de creación de una planificación, el sistema integra de forma nativa la opción de crear un movimiento personalizado si este no existe en la base de datos.
- **Catálogo de ejercicios:** Módulo orientado a la consulta y organización. Permite al usuario Ver ejercicios, Buscar / filtrar por grupos musculares, Crear ejercicio personalizado y Marcar favorito para un acceso rápido.
- **Sesión de entrenamiento:** Es el motor central de la aplicación durante la actividad física. Comienza con Iniciar entrenamiento, un caso de uso que incluye obligatoriamente (**<<include>>**) la acción de Registrar serie, ya que un entrenamiento no tiene validez sin el registro de cargas. A su vez, el registro de una serie cuenta con una funcionalidad opcional (**<<extend>>**) que permite Usar temporizador de descanso, mejorando la experiencia del usuario sin bloquear el flujo principal. El proceso culmina al Finalizar sesión.
- **Historial y progreso:** Módulo analítico de la plataforma. La acción de Ver historial requiere integrar el desglose de los datos (**<<include>>** hacia Ver detalle sesión). Del mismo modo, la acción de Registrar medidas corporales incluye directamente la visualización de la progresión mediante Ver gráficos.
- **Perfil de usuario:** Engloba la configuración personal, permitiendo Cambiar objetivo deportivo y Editar perfil. Esta última acción puede ser ampliada de forma opcional (**<<extend>>**) mediante la funcionalidad de Seleccionar foto para personalizar el avatar.

En conclusión, este diagrama refleja un sistema altamente cohesivo donde los flujos de trabajo están diseñados para reducir la fricción del usuario, automatizando los procesos intermedios (mediante includes) y ofreciendo herramientas de apoyo bajo demanda (mediante extends)

Diagrama Entidad-Relación (E-R)



El diagrama Entidad-Relación define el modelo conceptual y lógico de la base de datos de *FitProgress*, estructurado para garantizar la integridad referencial y evitar la redundancia de datos.

El esquema se compone de las siguientes entidades y relaciones cardinales:

- **USUARIOS:** Entidad pivotante del sistema. Posee relaciones de uno a muchos (1:N) con las entidades **`RUTINAS`** (un usuario puede diseñar múltiples rutinas), **`SESIONES_ENTRENAMIENTO`** (un usuario registra su histórico de entrenamientos), **`MEDIDAS_CORPORALES`** (registro biométrico a lo largo del tiempo) y **`EJERCICIOS_FAVORITOS`.**

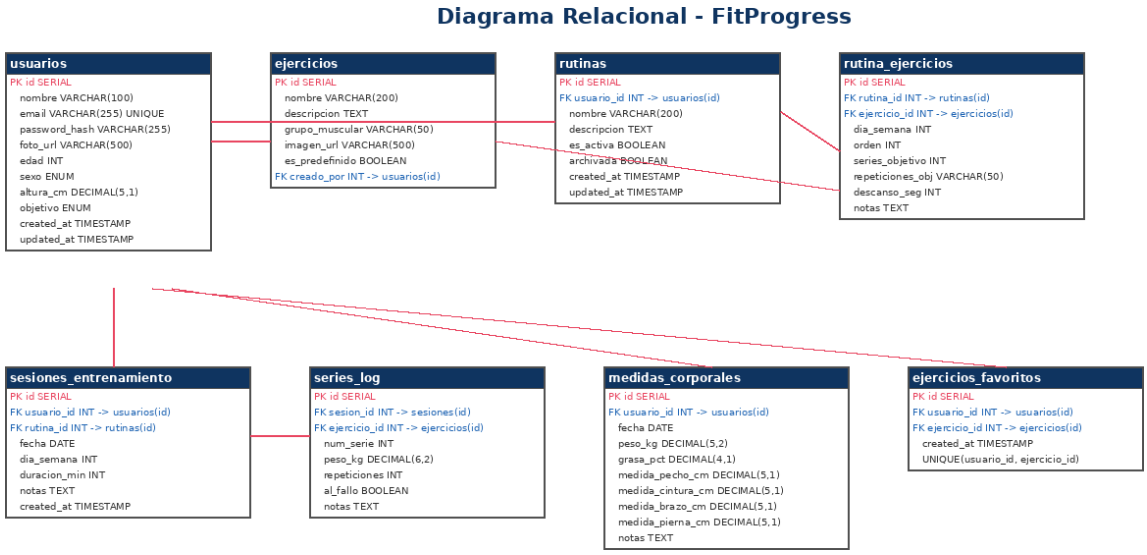
- **EJERCICIOS:** Almacena el catálogo de movimientos. Se relaciona de uno a muchos (1:N) con **`USUARIOS`** a través de la clave foránea **`FK creado_por`**, permitiendo identificar si el ejercicio es predefinido del sistema o personalizado por un usuario.

- **RUTINAS y RUTINA_EJERCICIOS:** Refleja una relación de muchos a muchos (N:M) entre **`RUTINAS y EJERCICIOS`**. Esta se rompe mediante la entidad asociativa **`RUTINA_EJERCICIOS`**, la cual no solo vincula ambas tablas, sino que almacena atributos críticos de la planificación: el día de la semana, el orden de

ejecución, las series objetivo, las repeticiones planificadas y los segundos de descanso.

- **SESIONES_ENTRENAMIENTO y SERIES_LOG:** Cuando el usuario entrena, se instancia una fila en **`SESIONES_ENTRENAMIENTO`** (vinculada opcionalmente a una `RUTINA` con cardinalidad 1:N). Cada ejercicio realizado genera múltiples filas en la entidad **`SERIES_LOG`**, la cual mantiene una relación de uno a muchos (1:N) dependiente de la sesión y del ejercicio concreto, almacenando el número de serie, el peso en kilogramos, las repeticiones y el indicador booleano de fallo muscular.

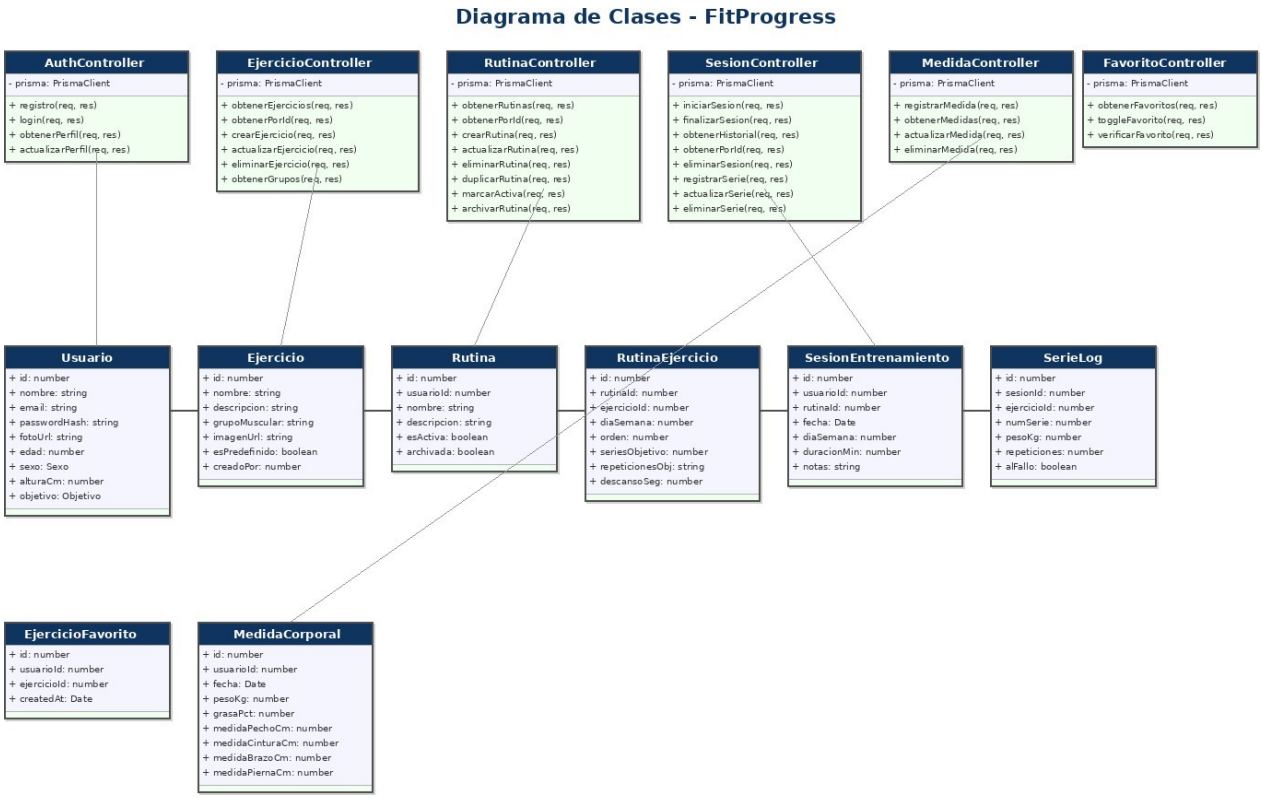
Diagrama Relacional



El diagrama relacional representa la implementación física de la base de datos en PostgreSQL, detallando las restricciones de Clave Primaria (PK), Clave Foránea (FK) y los tipos de datos asignados a nivel de almacenamiento.

Las claves foráneas (**`FK`**) aseguran la consistencia de los datos mediante restricciones de integridad. Destaca la configuración de borrado en cascada (*On Delete Cascade*) en las tablas dependientes como **`rutina_ejercicios`** y **`series_log`**; esto garantiza que si un usuario elimina una rutina o una sesión de entrenamiento, los registros huérfanos asociados se eliminen automáticamente, manteniendo la base de datos limpia y optimizada.

Diagrama de Clases

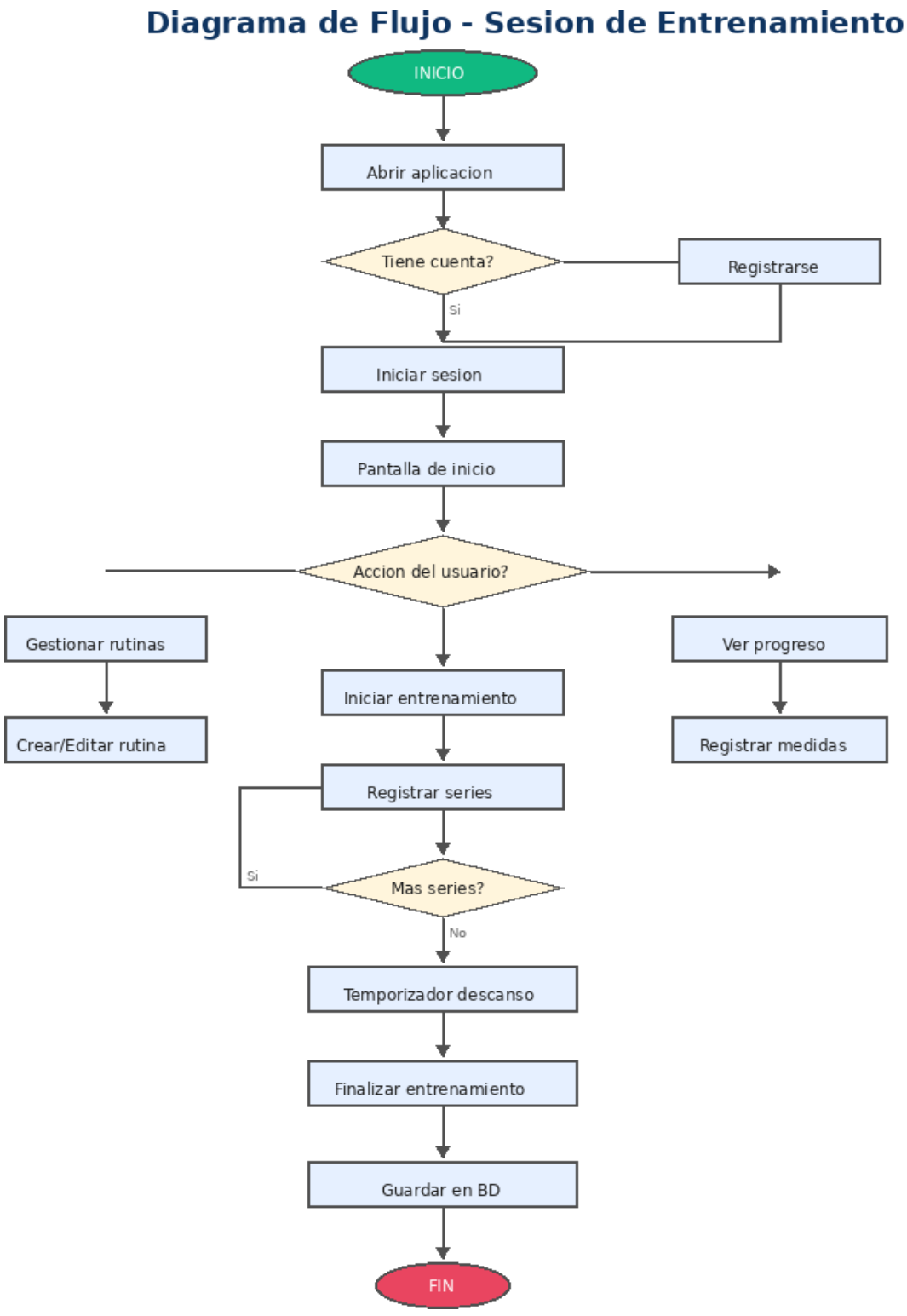


El diagrama de clases modela la estructura lógica del código en el servidor (*backend*), ilustrando el patrón arquitectónico Modelo-Vista-Controlador (MVC) implementado mediante Express y Prisma ORM.

En la capa superior se sitúan las clases controladoras (AuthController, EjercicioController,RutinaController,SesionController,MedidaController). Cada controlador expone métodos públicos (operaciones CRUD) que coinciden de forma exacta con los *endpoints* de las rutas de la API. Todos los controladores dependen de una instancia única del cliente de Prisma (prisma: PrismaClient) para comunicarse con la base de datos.

En la capa inferior se encuentran las clases de dominio o modelos de datos (Usuario, Ejercicio, Rutina, RutinaEjercicio, SesionEntrenamiento, SerieLog, EjercicioFavorito y MedidaCorporal), las cuales representan las estructuras y los tipos de TypeScript autogenerados por Prisma. Las líneas de asociación reflejan cómo los controladores procesan las peticiones HTTP e instancian o modifican los objetos de estas.

Diagrama de Flujo



El diagrama de flujo modela el aspecto algorítmico y el ciclo de vida operativo de una "Sesión de Entrenamiento" dentro de la aplicación, detallando la lógica secuencial y las bifurcaciones de decisión del sistema.

El flujo comienza con el inicio de la aplicación, donde un bloque de decisión evalúa si el usuario dispone de una cuenta activa; en caso negativo, se deriva al flujo de *Registrarse*, y en caso afirmativo, se procede a *Iniciar sesión* para acceder a la *Pantalla de inicio*.

A partir de la pantalla principal, el algoritmo entra en un estado de espera de la acción del usuario, abriendo tres caminos posibles: la gestión de planificaciones (*Gestionar rutinas - Crear/Editar rutina*), la consulta de métricas (*Ver progreso - Registrar medidas*) o el núcleo operativo (*Iniciar entrenamiento*). Al iniciar el entrenamiento, el sistema entra en un bucle iterativo controlado por la decisión *"¿Más series?"*. Mientras existan series por realizar, el usuario introduce los datos (*Registrar serie*) y el sistema activa de forma automatizada el *Temporizador de descanso*. Una vez completadas todas las series, el usuario interactúa con *Finalizar entrenamiento*, lo que dispara la función del servidor encargada de consolidar y persistir toda la estructura de datos en la base de datos (*Guardar en BD*), finalizando el proceso de forma segura.

• Desarrollo del proyecto

El desarrollo de *FitProgress* ha sido concebido bajo una metodología de ingeniería de software que prioriza la robustez estructural y la calidad del código, entendiendo que un proyecto de esta envergadura no solo debe ser funcional, sino también intrínsecamente sostenible. La toma de decisiones a lo largo de las distintas fases de implementación ha estado fundamentada en un análisis exhaustivo de las necesidades reales del usuario final, trasladadas a una arquitectura técnica capaz de escalar y evolucionar conforme a las demandas de un mercado digital cada vez más exigente. Se ha evitado en todo momento el uso de soluciones de compromiso que sacrificaran la integridad del sistema, optando por una arquitectura de capas bien definida donde la separación de responsabilidades entre el cliente móvil y el servidor central constituye el eje rector de todo el trabajo.

La configuración del entorno de desarrollo ha sido un proceso meticuloso donde el uso de herramientas como TypeScript ha marcado la pauta, permitiendo un tipado estricto de los datos que viajan desde la base de datos PostgreSQL hasta los componentes de React Native. Esta decisión ha resultado fundamental para minimizar la superficie de error, permitiendo detectar inconsistencias en las interfaces de datos mucho antes de que el código llegue a producción. La estructura de carpetas implementada no es un mero contenedor de archivos, sino un reflejo del patrón de arquitectura limpia. Mientras que en el frontend los componentes de interfaz han sido desacoplados para fomentar una reutilización extrema de los elementos visuales, en el backend la organización de los controladores responde a un criterio de cohesión funcional, donde cada módulo de código encapsula exclusivamente la lógica relacionada con su dominio, ya sea la gestión de usuarios, el historial de sesiones o el complejo motor de rutinas.

Un aspecto crítico del desarrollo ha sido la implementación del sistema de persistencia de datos. Tras evaluar diversas alternativas, se ha concluido que el uso de Prisma ORM sobre una base de datos relacional PostgreSQL era la solución óptima. Esta elección técnica ha permitido gestionar con elegancia relaciones complejas, como la cardinalidad de muchos a muchos existente entre rutinas y ejercicios, resolviéndola mediante tablas pivote que almacenan atributos dinámicos del entrenamiento, como el descanso o las repeticiones objetivo. A diferencia de las soluciones NoSQL, que habrían dificultado la consistencia de los datos en un entorno donde el historial médico y de rendimiento debe ser inmutable y preciso, este modelo relacional permite garantizar que cada serie registrada esté vinculada inequívocamente a una sesión, un ejercicio y un usuario, dotando al sistema de una integridad referencial absoluta.

La seguridad del sistema no se ha tratado como una capa añadida, sino como un elemento transversal desde la primera línea de código. El despliegue de un mecanismo de autenticación basado en tokens JWT ha permitido construir una API sin estado, lo que facilita que el servidor pueda gestionar de forma eficiente un elevado volumen de peticiones simultáneas sin los costes de memoria asociados a las sesiones tradicionales. La implementación de capas intermedias de middleware en Express no solo verifica la validez criptográfica de cada acceso, sino que también actúa como una barrera preventiva mediante validaciones exhaustivas de los datos entrantes, asegurando que ningún dato mal formado o malintencionado alcance la capa de persistencia de datos.

En lo referente a la experiencia de usuario y el motor de cálculo del entrenamiento, se han aplicado

optimizaciones que elevan la calidad de la aplicación por encima de las soluciones comerciales estándar. Al delegar el procesamiento analítico del volumen de trabajo y las cargas máximas al servidor, se ha logrado que la aplicación móvil se mantenga ligera y reactiva, incluso en dispositivos con hardware limitado. El flujo de entrenamiento activo demuestra esta optimización, donde la combinación de peticiones asíncronas para el guardado de series y la gestión local de temporizadores de descanso proporciona una respuesta instantánea al deportista. Cada serie registrada se procesa mediante una llamada segura al endpoint correspondiente, asegurando que el progreso del usuario esté protegido ante cualquier eventualidad técnica, mientras que el diseño de interfaces ha sido refinado iterativamente para que la interacción durante el entrenamiento sea mínima, intuitiva y eficaz.

Finalmente, el proceso de desarrollo ha culminado con una fase de pruebas funcionales y de estrés que ha permitido verificar cada uno de los casos de uso descritos en el apartado anterior. Estas pruebas no se han limitado a verificar la correcta ejecución de las rutas, sino que han buscado estresar el sistema ante escenarios de error, como la pérdida de conectividad durante el registro o el intento de acceso a recursos sin privilegios. El éxito en la superación de estas pruebas, junto con la correcta modularización y la documentación técnica exhaustiva que acompaña a cada controlador y componente, valida la calidad técnica del proyecto, consolidando a *FitProgress* como una aplicación profesional, escalable y preparada para su despliegue en entornos reales.

• Manual del Administrador

El despliegue y la administración de *FitProgress* han sido diseñados bajo una premisa de simplicidad operativa y reproducibilidad técnica. Con el fin de evitar procesos de ingeniería inversa o configuraciones manuales tediosas, el proyecto incorpora un sistema automatizado de migraciones y semillas de datos que permite levantar una instancia completa del servidor y la base de datos en cuestión de minutos, garantizando que cualquier entorno de pruebas sea idéntico al entorno de desarrollo original.

Para iniciar el despliegue del sistema, es requisito indispensable disponer de un entorno de ejecución basado en Node.js, preferiblemente en su versión LTS, y una instancia activa del sistema de gestión de bases de datos PostgreSQL. La comunicación entre el servidor y la persistencia se gestiona a través de Prisma ORM, lo que facilita enormemente la sincronización del esquema relacional. Una vez clonado el repositorio, el primer paso consiste en instalar todas las dependencias necesarias mediante el gestor de paquetes npm, un proceso que descarga automáticamente todas las librerías requeridas tanto para el backend como para las utilidades de migración.

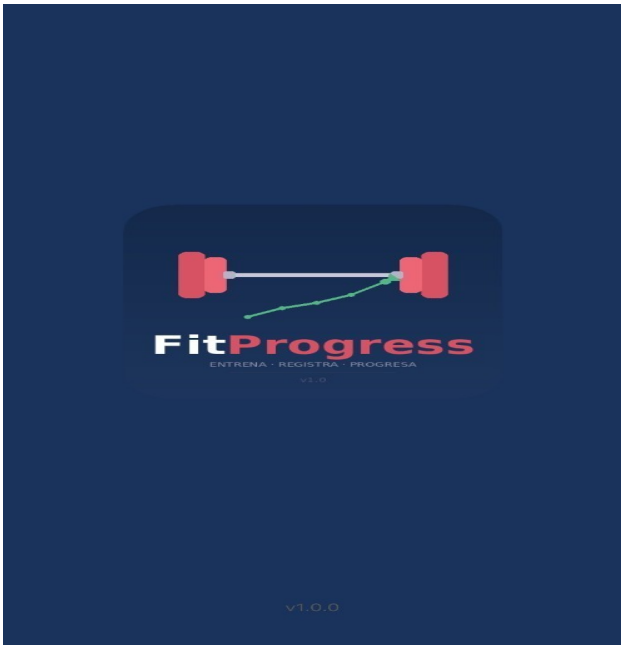
La configuración del entorno se centraliza en un archivo de variables de entorno, donde el administrador debe definir los parámetros críticos de conexión, tales como la cadena de conexión a la base de datos, el puerto de escucha del servidor y la clave secreta utilizada para la firma de tokens JWT. Para asegurar que este proceso sea sencillo, se ha incluido una plantilla de configuración que el administrador simplemente debe replicar y personalizar. Una vez definidos estos parámetros, la puesta en marcha de la base de datos es totalmente automática. Ejecutando el comando de migración de Prisma, el sistema construye de forma autónoma el esquema relacional completo en PostgreSQL, creando todas las tablas, relaciones y restricciones de integridad necesarias sin intervención manual.

Para completar la configuración, el sistema dispone de un script de semillado (*seed*) diseñado para cargar datos de prueba representativos. Este script no solo puebla las tablas básicas de usuarios y sesiones, sino que también inyecta el catálogo completo de ejercicios con sus descripciones técnicas y enlaces multimedia. Al ejecutar este script, el entorno de pruebas queda configurado inmediatamente con un conjunto de datos coherente, permitiendo que el evaluador pueda navegar por la aplicación y probar sus funcionalidades críticas desde el primer instante, sin necesidad de generar registros manualmente.

Respecto al cliente móvil, su ejecución sigue un flujo de trabajo estándar utilizando el CLI de Expo. Tras instalar las dependencias locales en el directorio del cliente, es fundamental que el administrador ajuste la constante de conexión a la API en el servicio de red para apuntar a la dirección IP local de la máquina que ejecuta el servidor. Este paso es el único requerimiento de configuración manual, dado que permite que el emulador o el dispositivo móvil real establezcan una comunicación bidireccional con la API RESTful. Una vez ajustada esta ruta, el lanzamiento del servidor de desarrollo de Expo CLI permite generar un código QR que, mediante la aplicación Expo Go, despliega la interfaz de usuario en cualquier dispositivo, completando así el ecosistema de *FitProgress* de forma ágil, profesional y estrictamente documentada.

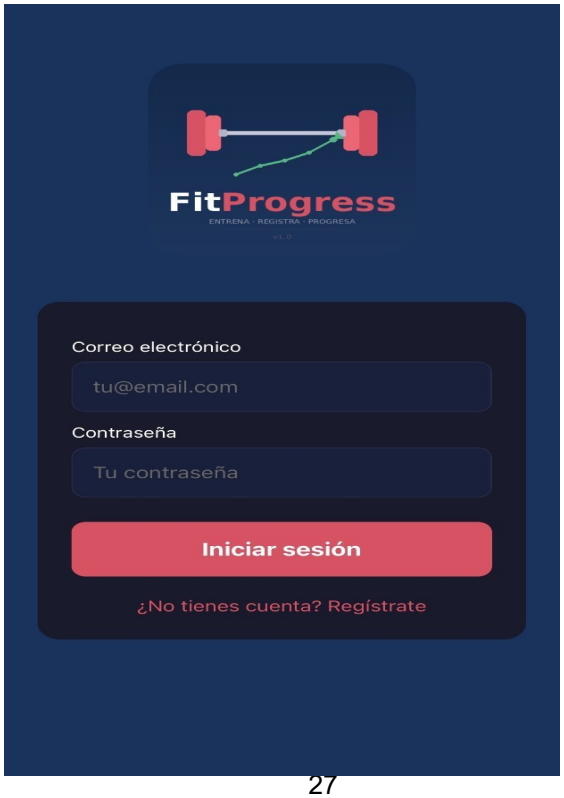
• Manual del Usuario

Pantalla de bienvenida



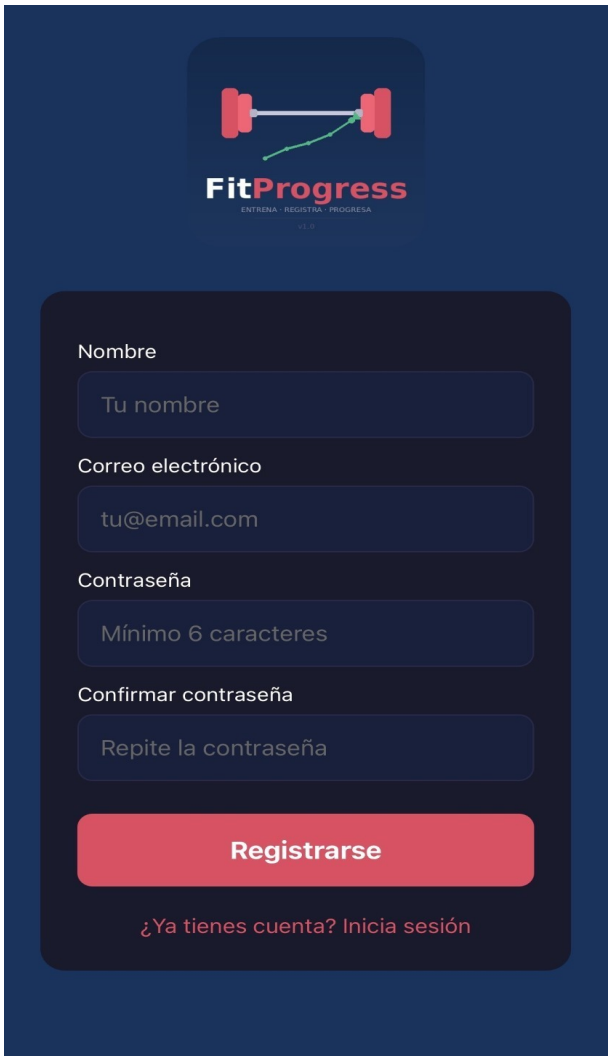
Al abrir la aplicación se muestra una pantalla de bienvenida con el logotipo de FitProgress y una animación de aparición. Esta pantalla se mantiene durante unos segundos antes de dar paso a la pantalla de inicio de sesión o a la pantalla principal si el usuario ya tiene una sesión activa.

Inicio de sesión



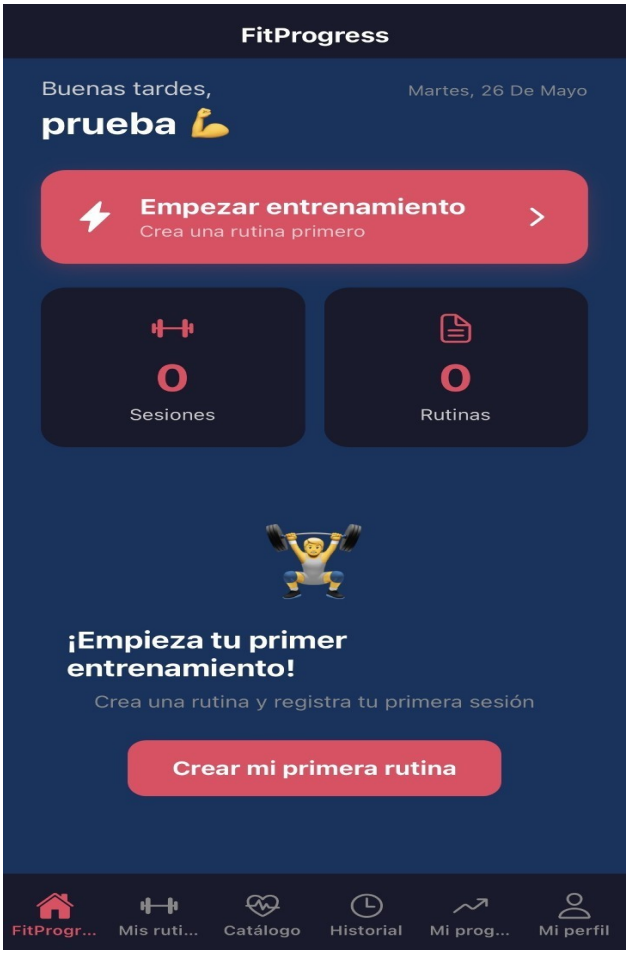
La pantalla de inicio de sesión presenta el logotipo de la aplicación seguido de un formulario con dos campos: correo electrónico y contraseña. El botón "Iniciar sesión" envía las credenciales al servidor, que las verifica y devuelve un token JWT que se almacena en el dispositivo para mantener la sesión activa. En la parte inferior, un enlace permite acceder al formulario de registro para los usuarios nuevos.

Registro de cuenta



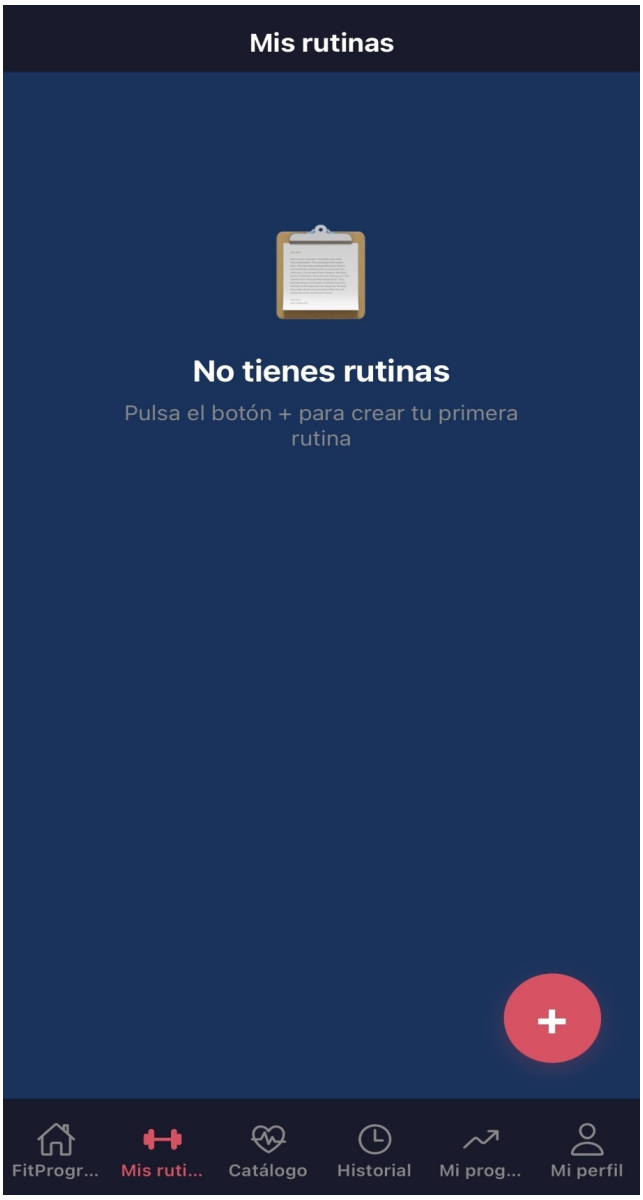
El formulario de registro solicita el nombre del usuario, correo electrónico, contraseña y confirmación de la contraseña. El sistema valida que el email no esté ya registrado, que la contraseña tenga al menos 6 caracteres y que ambas contraseñas coincidan. Una vez completado el registro, el usuario accede directamente a la aplicación sin necesidad de iniciar sesión manualmente.

Pantalla de inicio



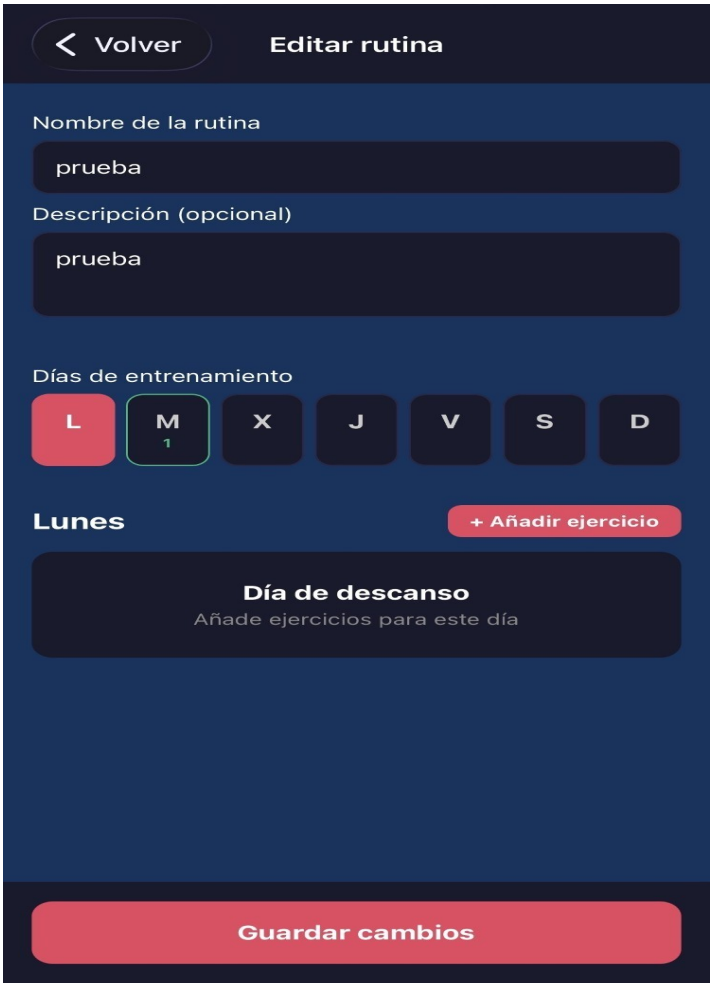
La pantalla de inicio funciona como el panel principal de la aplicación. En la parte superior muestra un saludo personalizado con el nombre del usuario y la fecha actual. Debajo se sitúa el botón destacado "Empezar entrenamiento", que detecta automáticamente la rutina activa y los ejercicios programados para el día de la semana en curso, permitiendo iniciar el entrenamiento con un solo toque. Más abajo se muestran dos tarjetas con estadísticas resumidas del número total de sesiones realizadas y rutinas creadas. Si el usuario ha realizado entrenamientos anteriores, se muestra una tarjeta con la información del último entrenamiento registrado.

Gestión de rutinas



La vista de rutinas presenta un listado de todas las rutinas creadas por el usuario. Cada tarjeta muestra el nombre de la rutina, una fila de indicadores con los días de la semana donde los días con ejercicios programados aparecen resaltados, y el número total de ejercicios incluidos. La rutina marcada como activa se distingue con un borde de color. En la parte inferior de cada tarjeta se despliegan botones de acción que permiten entrenar con esa rutina, activarla, duplicarla, archivarla o eliminarla. Un botón flotante en la esquina inferior derecha permite crear una nueva rutina.

Crear y editar rutina



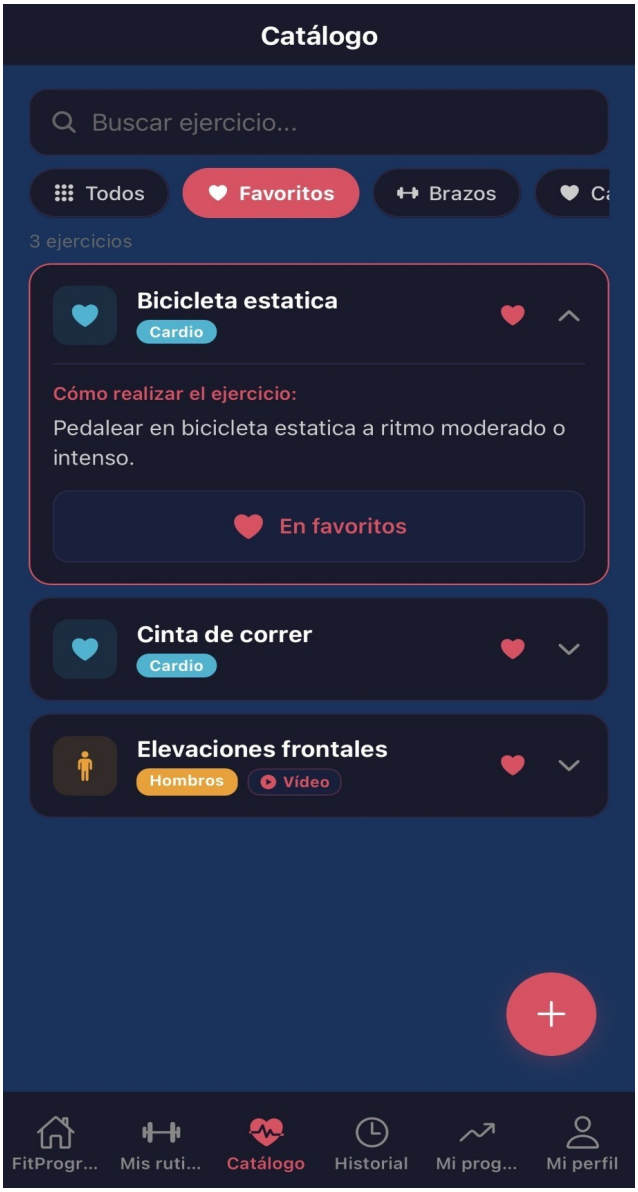
La pantalla de creación de rutina permite al usuario definir un nombre y una descripción para la planificación. Debajo se muestran los siete días de la semana como botones seleccionables. Al pulsar un día, se pueden añadir ejercicios del catálogo mediante un modal de selección con búsqueda. Para cada ejercicio añadido, el usuario puede configurar el número de series objetivo, las repeticiones planificadas y el tiempo de descanso entre series en segundos.

Catálogo de ejercicios



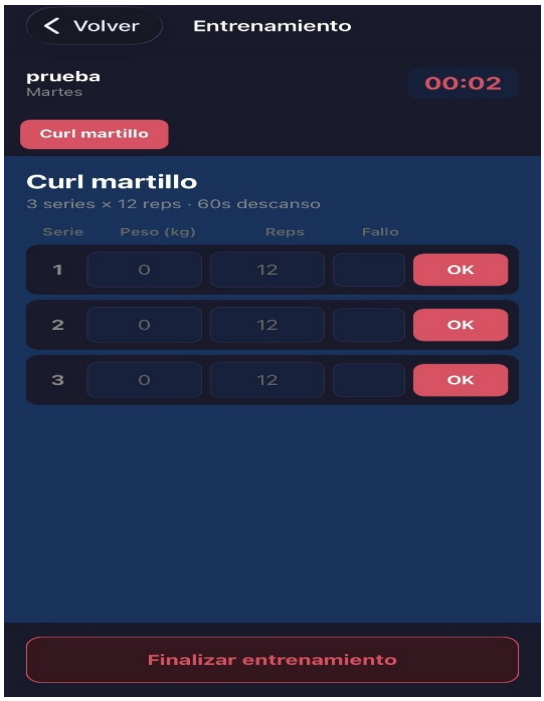
El catálogo de ejercicios muestra los 47 ejercicios predefinidos del sistema junto con los ejercicios personalizados creados por el usuario. En la parte superior se encuentra una barra de búsqueda y una fila de filtros por grupo muscular (pecho, espalda, hombros, brazos, piernas, core y cardio), además de un filtro de favoritos. Cada ejercicio se presenta como una tarjeta con su nombre, grupo muscular identificado con un color y un icono correspondiente. Al pulsar sobre un ejercicio, la tarjeta se expande mostrando la descripción de la técnica de ejecución, un botón para añadir o quitar de favoritos y un botón para abrir el vídeo demostrativo en YouTube.

Ejercicios favoritos



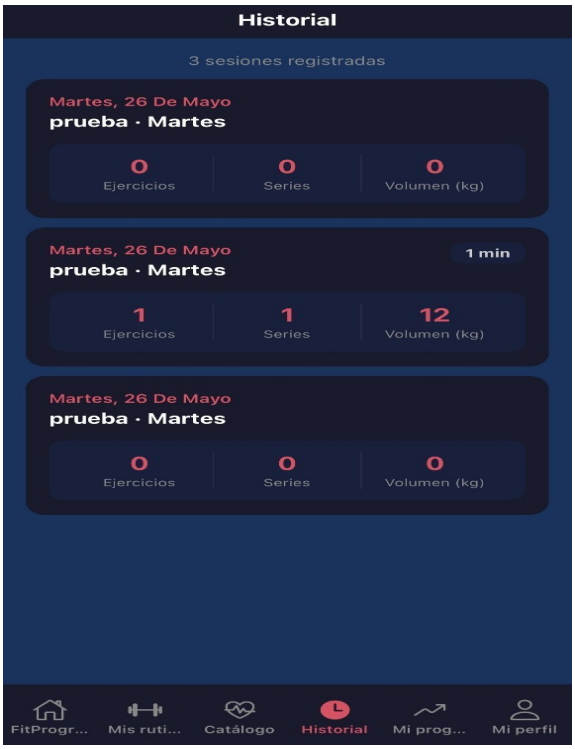
Al pulsar el filtro de "Favoritos" en el catálogo, la lista se filtra mostrando únicamente los ejercicios que el usuario ha marcado previamente con el corazón. Si no hay favoritos, se muestra un mensaje indicando que no se han añadido ejercicios a la lista.

Entrenamiento en vivo



La pantalla de entrenamiento en vivo es la más completa de la aplicación. En la parte superior muestra el nombre de la rutina, el día de entrenamiento y un cronómetro con el tiempo total de la sesión. Un selector horizontal permite cambiar entre los ejercicios programados para ese día, indicando con un color verde los que ya se han completado. Para cada ejercicio, se muestra una tabla donde el usuario introduce el peso en kilogramos y las repeticiones realizadas en cada serie, con un selector opcional para indicar si ha llegado al fallo muscular. Al confirmar una serie, se activa automáticamente un temporizador de descanso en la parte superior de la pantalla, con una cuenta atrás configurable que emite una vibración al finalizar. El botón de "Saltar descanso" permite continuar sin esperar. Al completar todos los ejercicios, el botón "Finalizar entrenamiento" guarda la sesión completa en la base de datos.

Historial de sesiones



El historial muestra un listado cronológico de todas las sesiones de entrenamiento realizadas. Cada tarjeta indica la fecha de la sesión, el nombre de la rutina utilizada, la duración en minutos y tres métricas resumidas: número de ejercicios, número de series y volumen total en kilogramos. Al pulsar sobre una sesión se accede a su detalle completo.

Detalle de sesión



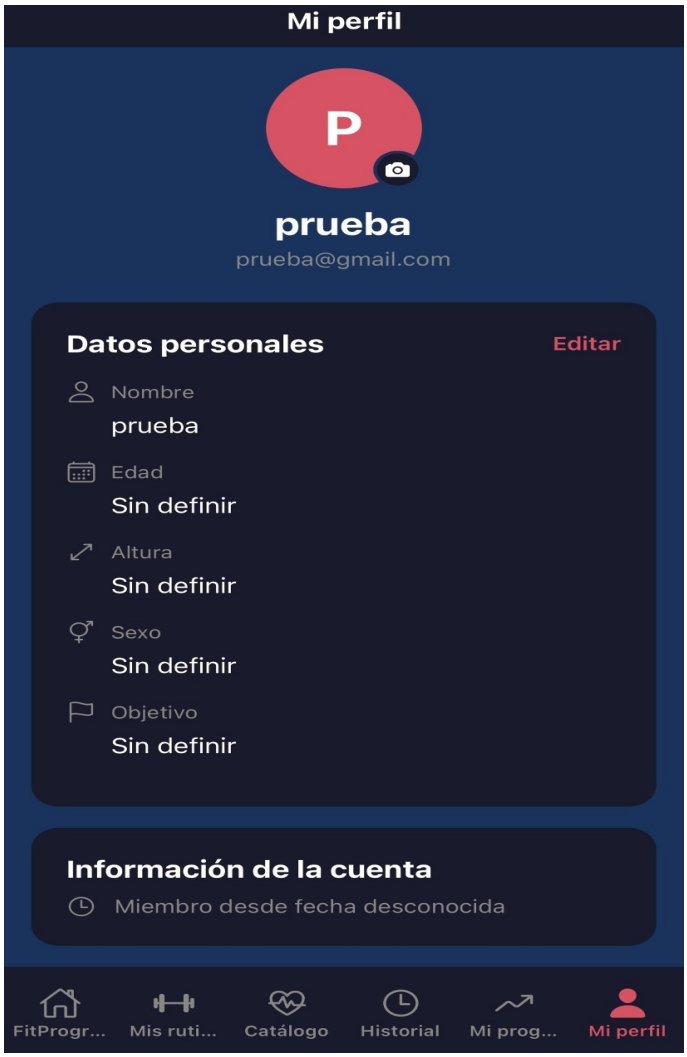
La vista de detalle de sesión muestra toda la información registrada durante un entrenamiento. En la cabecera aparece la fecha, el nombre de la rutina y las estadísticas generales. Debajo, cada ejercicio realizado se presenta con su tabla de series, indicando el peso, las repeticiones y si se alcanzó el fallo muscular, junto con el peso máximo y el volumen total por ejercicio.

Progreso y medidas corporales



La pantalla de progreso se divide en dos pestañas: "Medidas" y "Fuerza". En la pestaña de medidas, el usuario puede registrar sus medidas corporales mediante un formulario con campos para peso, porcentaje de grasa, y circunferencias de pecho, cintura, brazo y pierna. La evolución del peso se muestra mediante un gráfico de barras y el historial de medidas muestra cada registro con indicadores de cambio respecto a la medición anterior. En la pestaña de fuerza, se visualiza el volumen total de entrenamiento por sesión mediante barras horizontales que permiten comparar el rendimiento entre sesiones.

Perfil de usuario



La pantalla de perfil muestra la información personal del usuario con su inicial como avatar o la foto seleccionada desde la galería del dispositivo. Los datos personales (nombre, edad, altura, sexo y objetivo de entrenamiento) se pueden editar pulsando el botón "Editar" y guardando los cambios. Se muestra también la fecha de registro en la plataforma y un botón de cierre de sesión que requiere confirmación antes de ejecutarse.

Forma jurídica y trámites de constitución

Para la puesta en marcha del proyecto *FitProgress*, se ha optado por la forma jurídica de empresario individual (autónomo). Esta elección se justifica por varios motivos: en primer lugar, supone menores costes iniciales, ya que no requiere la aportación de un capital social mínimo, permitiendo destinar los recursos a la infraestructura tecnológica (servidores y licencias de desarrollo); en segundo lugar, ofrece una gestión administrativa mucho más sencilla y flexible; y, finalmente, se adapta de manera idónea a un proyecto de software a pequeña escala en el que la actividad depende principalmente del trabajo personal del promotor. Aunque la responsabilidad es ilimitada, el nivel de riesgo económico y operativo asociado a una aplicación móvil es moderado, por lo que esta forma jurídica resulta plenamente adecuada.

Los principales trámites necesarios para la constitución como autónomo son los siguientes:

- **Alta en la Agencia Tributaria:** Mediante la presentación del modelo 036 o 037, indicando el inicio de actividad y el epígrafe del IAE correspondiente (relacionado con el desarrollo de software o servicios informáticos).
- **Alta en la Seguridad Social:** Inscripción en el Régimen Especial de Trabajadores Autónomos (RETA).
- **Licencias municipales:** Al tratarse de un producto digital (*Software as a Service*), el desarrollo puede realizarse en remoto. No obstante, en caso de desarrollar la actividad en un local físico o un espacio de trabajo propio, se procederá a la obtención de la licencia de apertura o comunicación previa en el ayuntamiento correspondiente.
- **Gestión administrativa:** Cumplimiento de las obligaciones fiscales y contables básicas, como la liquidación del IVA o el IRPF correspondientes a la comercialización de la plataforma.

• Viabilidad tecno-económica

El presente apartado analiza los costes asociados al desarrollo e implantación del proyecto FitProgress, considerando tanto los recursos de hardware como de software empleados, así como los costes de personal y de infraestructura necesarios para su puesta en producción.

Costes de desarrollo

Para el desarrollo del proyecto se ha utilizado un equipo personal y herramientas de software gratuitas o de código abierto, lo que ha permitido minimizar significativamente los costes iniciales. El tiempo de desarrollo estimado ha sido de aproximadamente 4 semanas a jornada parcial, equivalente a unas 120 horas de trabajo.

Costes de hardware

El desarrollo se ha realizado con un ordenador portátil personal y un dispositivo móvil para las pruebas de la aplicación a través de Expo Go. No ha sido necesario adquirir hardware adicional.

Concepto	Coste
Ordenador portátil (amortización proporcional)	150,00 €
Dispositivo móvil para pruebas (amortización proporcional)	50,00 €
Subtotal hardware	200,00 €

Fuente: Elaboración propia.

Costes de software

Todas las tecnologías utilizadas en el proyecto son de código abierto o disponen de planes gratuitos, por lo que el coste de licencias de software es nulo.

Concepto	**Coste**
Node.js (open source)	0,00 €
React Native + Expo (open source)	0,00 €
PostgreSQL (open source)	0,00 €
Prisma ORM (open source)	0,00 €
Visual Studio Code (gratuito)	0,00 €
GitHub (plan gratuito)	0,00 €
pgAdmin 4 (open source)	0,00 €
Subtotal software	**0,00 €**

Fuente: Elaboración propia

Costes de personal

Se estima el coste de un desarrollador junior a 15 €/hora, teniendo en cuenta que se trata de un proyecto de fin de ciclo formativo.

Concepto	Horas	Coste/hora	Coste total
Análisis y diseño	15 h	15,00 €	225,00 €
Desarrollo backend (API + BD)	40 h	15,00 €	600,00 €
Desarrollo frontend (app móvil)	45 h	15,00 €	675,00 €
Pruebas y corrección de errores	10 h	15,00 €	150,00 €
Documentación y memoria	10 h	15,00 €	150,00 €
Subtotal personal	**120 h**		**1.800,00 €**

Fuente: Elaboración propia

Costes de infraestructura (producción)

En caso de querer publicar la aplicación para uso real, se necesitaría un servidor en la nube y una cuenta de desarrollador para las tiendas de aplicaciones.

Concepto	Coste mensual	Coste anual
Servidor VPS (DigitalOcean, 1 GB RAM)	6,00 €	72,00 €
Dominio web	1,00 €	12,00 €
Cuenta Google Play Store (pago único)	—	25,00 €
Cuenta Apple Developer	—	99,00 €
Subtotal infraestructura (primer año)		**208,00 €**

Fuente: Elaboración propia

Tabla resumen de costes totales

Categoría	Coste
Hardware	200,00 €
Software	0,00 €
Personal	1.800,00 €
Infraestructura (primer año)	208,00 €
COSTE TOTAL DEL PROYECTO	**2.208,00 €**

Fuente: Elaboración propia

Análisis de rentabilidad

El proyecto podría resultar rentable mediante un modelo de suscripción freemium. La versión gratuita incluiría las funcionalidades básicas de creación de rutinas y registro de entrenamientos, mientras que la versión premium ofrecería funcionalidades avanzadas como gráficos de evolución detallados, exportación de datos y rutinas prediseñadas por profesionales.

Con una cuota mensual de 2,99 € para la versión premium, se necesitarían aproximadamente 62 suscriptores mensuales para cubrir los costes del primer año. Considerando que aplicaciones similares en el mercado cuentan con miles de usuarios, se estima que el proyecto sería viable económicamente a medio plazo.

Además, el uso exclusivo de tecnologías de código abierto permite mantener los costes operativos muy bajos, lo que facilita alcanzar el punto de equilibrio con un número reducido de usuarios de pago.

• Trabajo futuro

La culminación de la primera fase de desarrollo de *FitProgress* establece una arquitectura sólida y funcional que, no obstante, representa solo el cimiento de un ecosistema tecnológico diseñado para la evolución constante. Las líneas de trabajo futuro se orientan hacia la expansión de la resiliencia técnica, la integración profunda en el ecosistema de dispositivos vestibles y la implementación de mecanismos de análisis de datos que transformen la aplicación de una herramienta de registro en un asistente proactivo. En el plano de la seguridad y la integridad de las cuentas, se proyecta el desarrollo de un módulo de verificación de identidad mediante el servicio SMTP empleando la librería *Nodemailer*. Esta implementación permitirá transitar hacia un modelo de registro verificado, donde cada usuario deba validar su dirección de correo electrónico a través de un enlace securizado, elevando los estándares de seguridad y reduciendo la creación de cuentas sintéticas o automatizadas. Acompañando a esta mejora, se contempla la introducción de la autenticación de doble factor y la recuperación de credenciales mediante canales cifrados, consolidando la confianza del usuario en el tratamiento de su información personal.

En términos de interacción y retención, la hoja de ruta técnica prioriza la integración de servicios de mensajería en la nube mediante *Firebase Cloud Messaging*. Esta funcionalidad permitirá al servidor emitir notificaciones proactivas al dispositivo móvil para incentivar la adherencia al plan de entrenamiento, alertar sobre hitos de progreso alcanzados o recordar sesiones programadas. Para sustentar esta interactividad, se hace indispensable evolucionar la arquitectura actual hacia un modelo estrictamente *offline-first*. La futura implementación de motores de sincronización híbrida y bases de datos locales optimizadas para dispositivos móviles, como *WatermelonDB*, permitirá al deportista operar en entornos de baja o nula conectividad garantizando que cada serie registrada se persista localmente y se sincronice con el servidor PostgreSQL de forma transparente y resiliente en cuanto se detecte una conexión estable. Esta mejora técnica se complementará con la expansión hacia el ecosistema de *wearables*, desarrollando capas de abstracción para las APIs nativas de *Apple HealthKit* y *Google Fit*. La integración de estos datos permitirá que el sistema no solo registre el volumen de entrenamiento de fuerza, sino que lo correlacione con variables fisiológicas externas como la frecuencia cardíaca y el gasto calórico total, ofreciendo una visión holística del rendimiento del atleta.

Finalmente, la visión a largo plazo para *FitProgress* incluye la transformación de la plataforma en un espacio social y comercialmente viable. Se proyecta el desarrollo de un módulo de intercambio de rutinas que permita a los usuarios compartir sus planificaciones en un catálogo comunitario global, fomentando la competitividad saludable y el aprendizaje mutuo. Para sustentar la sostenibilidad económica del proyecto, se integrará una pasarela de pagos profesional mediante el SDK de *Stripe*, facilitando la transición hacia un modelo de negocio que combine la gratuidad de las funciones básicas con suscripciones premium para el acceso a planes de entrenamiento diseñados por especialistas. La culminación de estas etapas se verá potenciada por la incorporación de

modelos analíticos en el servidor para el estudio de tendencias, permitiendo que la aplicación detecte estancamientos de fuerza y sugiera ajustes automáticos en la carga o la selección de ejercicios basándose en el historial de rendimiento del usuario, posicionando a *FitProgress* como una solución líder, inteligente y tecnológicamente avanzada en el sector de la salud digital.

• Conclusiones

La finalización del desarrollo de *FitProgress* permite realizar un balance altamente positivo tanto desde la perspectiva de la funcionalidad del producto final obtenido como desde el prisma de la consolidación de competencias técnicas adquiridas a lo largo del Ciclo Formativo. En primer lugar, se puede afirmar con rotundidad que los objetivos generales y específicos propuestos al inicio del proyecto han sido alcanzados con éxito. El sistema se ha consolidado como una herramienta *Full-Stack* plenamente operativa que cubre con solvencia todo el ciclo de vida del software, ofreciendo una alternativa robusta, fluida y abierta frente a las soluciones restrictivas del mercado actual. Se ha logrado dotar al usuario de una interfaz móvil multiplataforma que minimiza la fricción en la entrada de datos durante la actividad física, estableciendo una comunicación segura, eficiente y desatendida con una API REST en la nube, garantizando en todo momento la integridad y persistencia de la información.

En el plano estrictamente tecnológico, el desarrollo ha servido para superar retos de ingeniería de software de alta complejidad. La gestión de la asincronía en las peticiones de red y el uso estricto de TypeScript en ambas capas del proyecto han demostrado ser decisiones acertadas, contribuyendo a la creación de un sistema resiliente, tipado y significativamente libre de errores en tiempo de ejecución. Asimismo, el modelado relacional implementado a través de Prisma ORM ha permitido resolver con elegancia cardinalidades complejas, asegurando una consistencia referencial impecable y un rendimiento óptimo de las consultas, incluso ante volúmenes de datos crecientes. No obstante, en este proceso de desarrollo se ha reflexionado sobre aspectos que, si bien han cumplido los requisitos, son susceptibles de evolución; por ejemplo, la gestión de la sincronización en entornos sin conexión, que aunque resuelta mediante persistencia en base de datos, abre la puerta a una arquitectura *offline-first* más ambiciosa en futuras iteraciones.

Más allá del cumplimiento de los objetivos técnicos, este proyecto reafirma el valor de las arquitecturas de software modernas, desacopladas y basadas en microservicios o servicios independientes. La combinación tecnológica seleccionada Node.js, Express, React Native y PostgreSQL representa un ecosistema de alta demanda en el mercado profesional debido a su escalabilidad y eficiencia operativa. La culminación de este trabajo no solo valida el esfuerzo realizado, sino que también sienta unas bases técnicas sólidas sobre las cuales desarrollar, en etapas posteriores, funcionalidades avanzadas como la integración de pasarelas de pago, el análisis predictivo del rendimiento mediante modelos de datos o la creación de redes comunitarias de entrenamiento. En conclusión, la defensa de esta memoria representa la consolidación de un perfil profesional capaz de abordar desafíos técnicos reales, afrontando con garantías los estándares de calidad y excelencia que exige el sector tecnológico actual.

• Biblioteca de recursos web y referencias

Meta Platforms, Inc. (2026): "Getting Started – React Native", documentación oficial del framework de desarrollo móvil multiplataforma, disponible en: https://reactnative.dev/docs/getting-started

Expo, Inc. (2026): "Expo Documentation", documentación oficial del entorno de desarrollo para aplicaciones React Native, disponible en: https://docs.expo.dev/

OpenJS Foundation (2026): "Node.js v22 Documentation", documentación oficial del entorno de ejecución de JavaScript en servidor, disponible en: https://nodejs.org/en/docs

StrongLoop, Inc. (2026): "Express 5.x – API Reference", documentación oficial del framework web minimalista para Node.js, disponible en: https://expressjs.com/en/5x/api.html

Prisma Data, Inc. (2026): "Prisma ORM Documentation – Quickstart with TypeScript & PostgreSQL", documentación oficial del ORM utilizado para la gestión de la base de datos y las migraciones, disponible en: https://www.prisma.io/docs/getting-started

The PostgreSQL Global Development Group (2026): "PostgreSQL 18 Documentation", documentación oficial del sistema de gestión de bases de datos relacional utilizado en el proyecto, disponible en: https://www.postgresql.org/docs/18/index.html

Microsoft Corporation (2026): "TypeScript Handbook", documentación oficial del lenguaje de tipado estático utilizado tanto en el frontend como en el backend del proyecto, disponible en: https://www.typescriptlang.org/docs/handbook/intro.html

Bibliotecas y paquetes utilizados

Satya, S. y colaboradores (2026): "React Navigation – Getting Started", documentación oficial de la librería de navegación utilizada para la gestión de pantallas y pestañas en la aplicación, disponible en: https://reactnavigation.org/docs/getting-started

Oblador, J. (2026): "react-native-vector-icons", repositorio oficial de la librería de iconos vectoriales para React Native, utilizada para los iconos Ionicons de la interfaz, disponible en: https://github.com/oblador/react-native-vector-icons

Kraus, T. y React Native Community (2026): "@react-native-async-storage/async-storage", documentación oficial de la librería de almacenamiento local asíncrono utilizada para la persistencia del token JWT en el dispositivo, disponible en: https://react-native-async-storage.github.io/async-storage/

Auth0, Inc. (2026): "node-jsonwebtoken", repositorio oficial de la librería utilizada para la generación y verificación de tokens de autenticación JWT, disponible en: https://github.com/auth0/node-jsonwebtoken

Kelektiv (2026): "node.bcrypt.js", repositorio oficial de la librería utilizada para el cifrado de contraseñas de los usuarios mediante el algoritmo bcrypt, disponible en:

https://github.com/kelektiv/node.bcrypt.js

Campos, G. (2026): "express-validator", documentación oficial del middleware de validación de datos de entrada en las rutas de la API REST, disponible en: https://express-validator.github.io/docs/

Expo, Inc. (2026): "ImagePicker – Expo SDK", documentación oficial del módulo utilizado para la selección de fotografías de perfil desde la galería del dispositivo, disponible en: https://docs.expo.dev/versions/latest/sdk/imagepicker/

Expo, Inc. (2026): "Linking – Expo SDK", documentación oficial del módulo utilizado para la apertura de enlaces externos a los vídeos demostrativos de YouTube, disponible en: https://docs.expo.dev/versions/latest/sdk/linking/

Recursos multimedia integrados en la aplicación

YouTube (2026): Vídeos demostrativos de técnica de ejecución de ejercicios de musculación. Se han integrado 43 enlaces a vídeos de acceso público alojados en la plataforma YouTube, accesibles desde el catálogo de ejercicios de la aplicación mediante la librería Expo Linking.

Recursos de consulta y resolución de problemas

Mozilla Foundation (2026): "MDN Web Docs – JavaScript Reference", referencia técnica del lenguaje JavaScript consultada durante el desarrollo, disponible en: https://developer.mozilla.org/en-US/docs/Web/JavaScript

Stack Overflow (2026): "Stack Overflow", plataforma de preguntas y respuestas de programación consultada para la resolución de errores técnicos puntuales durante el desarrollo del proyecto, disponible en: https://stackoverflow.com/

Herramientas de desarrollo y control de versiones

Microsoft Corporation (2026): "Visual Studio Code Documentation", documentación del editor de código utilizado como entorno de desarrollo principal, disponible en: https://code.visualstudio.com/docs

GitHub, Inc. (2026): "GitHub Docs", documentación de la plataforma utilizada para el alojamiento del repositorio y el control de versiones del proyecto mediante Git, disponible en: https://docs.github.com/

The PostgreSQL Global Development Group (2026): "pgAdmin 4 Documentation", documentación de la herramienta de administración visual utilizada para la gestión y consulta de la base de datos PostgreSQL, disponible en: https://www.pgadmin.org/docs/

Todas las imágenes, diagramas, tablas y gráficos incluidos en la presente memoria son de elaboración propia por parte del alumno, salvo indicación expresa mediante nota al pie de página.

• Anexos

Anexo I: Glosario de arquitectura técnica

Para asegurar una interpretación clara de los conceptos técnicos y tecnológicos empleados en el sistema, se definen los siguientes términos:

- **API RESTful (Representational State Transfer):** Paradigma de arquitectura que garantiza la comunicación entre el servidor y el cliente móvil mediante una interfaz desacoplada basada en el protocolo HTTP, utilizando los métodos estándar GET, POST, PUT y DELETE para la gestión de recursos.
- **Ciclo de vida asíncrono (Asynchronous I/O):** Modelo de ejecución fundamentado en Node.js que permite gestionar múltiples operaciones de entrada y salida sin bloqueo del hilo principal, lo que resulta esencial para el rendimiento de la API ante peticiones concurrentes de varios usuarios.
- **Cifrado unidireccional (bcrypt):** Mecanismo de hashing adaptable con sal (salting) empleado para la protección de las contraseñas de los usuarios, garantizando que el almacenamiento de credenciales sea seguro al ser computacionalmente inviable su reversión.
- **ORM (Object-Relational Mapping):** Capa de abstracción proporcionada por Prisma que permite interactuar con la base de datos PostgreSQL mediante modelos tipados en TypeScript, evitando la escritura directa de consultas SQL y minimizando errores en tiempo de ejecución.
- **Middleware de seguridad:** Capa intermedia de software que intercepta las peticiones entrantes en el servidor, validando la autenticidad del token de sesión (JWT) y comprobando los datos de entrada mediante express-validator antes de ejecutar cualquier lógica de negocio.
- **Arquitectura Stateless:** Configuración del sistema donde el servidor no mantiene el estado de la sesión entre peticiones, delegando la identificación del usuario en el token JWT cifrado que este envía en cada llamada a la API. Esto permite una escalabilidad eficiente del servidor.
- **Expo Go:** Aplicación móvil que permite ejecutar y probar proyectos desarrollados con React Native y Expo en dispositivos reales sin necesidad de compilar el código de forma nativa, agilizando el proceso de desarrollo y pruebas.
- **Migración de base de datos:** Proceso automatizado mediante Prisma que permite crear, modificar o eliminar tablas y relaciones en la base de datos PostgreSQL a partir del esquema definido en el archivo schema.prisma, manteniendo un historial versionado de los cambios.
- **Seed (semilla de datos):** Script que inserta datos iniciales en la base de datos para que la aplicación no arranque vacía. En este proyecto, el seed carga el catálogo de 47 ejercicios predefinidos con sus descripciones y enlaces a vídeos demostrativos.

Anexo II: Especificaciones de diseño relacional

La integridad de los datos en FitProgress se articula mediante un esquema de base de

datos relacional normalizado, diseñado para garantizar la consistencia referencial en todas las operaciones. Las entidades principales que componen la base de datos se definen bajo las siguientes premisas:

- **Entidad Usuario:** Núcleo de la plataforma, que actúa como entidad central del sistema. Utiliza identificadores autoincrementales (SERIAL) como clave primaria y garantiza la unicidad de las credenciales mediante una restricción UNIQUE en el campo de correo electrónico. Mantiene relaciones de uno a muchos con las entidades Rutina, Sesión, Medida Corporal, Ejercicio y Ejercicio Favorito.
- **Entidad Ejercicio:** Catálogo centralizado de movimientos clasificados por grupo muscular (pecho, espalda, hombros, brazos, piernas, core y cardio). Incluye un campo booleano que distingue los ejercicios predefinidos del sistema de los creados por el usuario, y un campo opcional para almacenar la URL del vídeo demostrativo de YouTube.
- **Entidad Rutina:** Estructurada bajo una relación de uno a muchos con el usuario, permite la creación de múltiples planificaciones de entrenamiento. Incluye indicadores de estado (activa/archivada) para la gestión del ciclo de vida de cada rutina.
- **Entidad Rutina_Ejercicios:** Tabla asociativa que resuelve la relación de muchos a muchos entre Rutinas y Ejercicios. Además de vincular ambas entidades, almacena atributos propios de la planificación: el día de la semana, el orden de ejecución, las series objetivo, las repeticiones planificadas y los segundos de descanso entre series.
- **Entidad Sesión_Entrenamiento:** Registra cada sesión de entrenamiento realizada por el usuario, vinculada opcionalmente a una rutina. Almacena la fecha, el día de la semana, la duración en minutos y posibles notas del deportista.
- **Entidad Series_Log:** Registro inmutable de cada serie ejecutada durante un entrenamiento. Vincula cada serie a una sesión y a un ejercicio concreto, almacenando el número de serie, el peso en kilogramos, las repeticiones realizadas y un indicador booleano de fallo muscular. La relación con la sesión incluye borrado en cascada para mantener la base de datos limpia.
- **Entidad Medida_Corporal:** Permite el seguimiento biométrico del usuario a lo largo del tiempo, registrando peso corporal, porcentaje de grasa y circunferencias de pecho, cintura, brazo y pierna.
- **Entidad Ejercicio_Favorito:** Tabla que almacena los ejercicios marcados como favoritos por cada usuario, con una restricción UNIQUE compuesta que impide duplicados (un usuario no puede marcar el mismo ejercicio como favorito dos veces).

Anexo III: Guía de configuración del entorno

Para asegurar que cualquier evaluador o administrador pueda desplegar una réplica exacta del entorno de desarrollo, se especifica la configuración necesaria del archivo de variables de entorno (.env) ubicado en la carpeta server del proyecto.

El archivo .env debe contener las siguientes variables:

- **DATABASE_URL:** Cadena de conexión a la base de datos PostgreSQL, que

incluye el usuario, la contraseña, la dirección del servidor, el puerto y el nombre de la base de datos. Ejemplo: `DATABASE_URL="postgresql://postgres:1234@127.0.0.1:5433/fitprogress"`

- **JWT_SECRET:** Clave secreta utilizada para la firma y verificación de los tokens de autenticación JWT. Debe ser una cadena lo suficientemente larga y compleja para garantizar la seguridad de las sesiones. Ejemplo: `JWT_SECRET="fitprogress_secreto_jwt_2026"`
- PORT: Puerto de escucha del servidor Express. Por defecto se utiliza el puerto 3000. Ejemplo: `PORT=3000`

Adicionalmente, la constante de conexión del frontend con el backend se encuentra en el archivo `app/src/services/api.ts`, donde debe configurarse la dirección IP de la máquina que ejecuta el servidor:

```
const API_URL = 'http://[IP_LOCAL]:3000/api';
```

Esta IP debe corresponder a la dirección IPv4 de la máquina en la red local, que puede consultarse ejecutando el comando `ipconfig` en Windows. Es imprescindible que el dispositivo móvil y el ordenador estén conectados a la misma red WiFi para que la comunicación entre el frontend y el backend funcione correctamente.